

Intelligent Robotics Assignment 1

Michele Zarantonello 2162530

Riccardo Grosso 2155345

Luca Cordes 2158346

{michele.zarantonello, riccardo.grosso.1, janlucamichael.cordes}@studenti.unipd.it

Github Repo Video

Github accounts: michelezarantonello, groricc, lucordesunipd

November 30, 2025

1 Introduction

Basic goal: The pipeline must then include:

- detection of the Apriltag
- navigation to the Apriltags (find a good position between the two and reach it without hitting them)
- detection of tables using any available sensor
- return the position of tables relative to odom reference frame

The groundwork for detecting the Apriltags was already done and can be found in the given `ir_2526` repo.

2 Detection of the Apriltag

Starting from the not-configured `apriltag_ros` package located in `~ws_group_32_assignments/src/ir_2526`, we applied several modifications to fit the specific requirements of our task. In particular, the Apriltag size was set to 0.05 m, and some topic remappings were necessary, such as: `image_rect` to `/rgb_camera/image` and `camera_info` to `/rgb_camera/camera_info`. After these changes, the camera was able to correctly detect the two Apriltags (IDs `tag36h11:1` and `tag36h11:10`) and publish the corresponding transformations.

3 Navigation to midpoint between Apriltags

To understand the following logic, one must refer to Figure 3.

Before starting the navigation, the lifecycle nodes manager must activate the nodes for the localization (in particular activating ...), making it possible to set the initial pose with respect to the "map" reference frame.

In our implementation, `manage_localization_nodes.cpp` was responsible for activating the nodes for localization while `init_pose_publisher.cpp` was (obviously) responsible for setting the initial pose. Once the initial pose is published in `/initialpose` topic another lifecycle node manager activates the nodes for the navigation (in particular). Now every node we need to reach the desired pose is active. Now `nav2pose.cpp` comes in, creating an action client that sends a goal request in `navigate_to_pose`. The goal pose was obtained by looking up the transforms between the map frame (global reference frame) and both Apriltags, and then computing their midpoint. Once the robot reaches the goal, `table_detection.cpp` will be run thus detecting the position of the three cylindrical tables and, by making use of the transformation, print on the console the position of them in odom reference frame.

4 Table Detection

This node is responsible for the detection of the tables in its surroundings using any available sensor. The node uses the laser Scanner sensor feedback to detect objects around the scanner. This sensor publishes a message into the `/scan` topic, the table detector is subscribed to that topic and is able to acquire and analyze the data.

4.1 Acquisition and Analysis

Data is acquired using a topic callback every time that a publisher sends data. The data is then processed, looping through all the ranges sensed by the laser scanner to find minimum clusters. An example of scan can be viewed in Figure 1, where the points of the tables scanned are highlighted by blue rectangles. These minimum clusters points are then used to estimate the radius of the tables, and so the coordinates w.r.t the laser reference frame. In the end the position of the tables are transformed w.r.t. the required `odom` reference frame. The important code of the table detector can be found in Appendix, section 7.1.

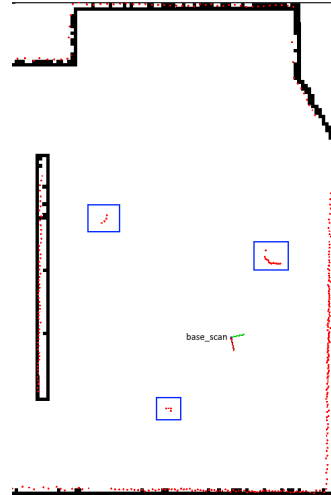


Figure 1: Laser scan viewed from RViz window (red dots).

5 Problems

During the process of solving the assignment we have encountered many problems.

- We initially implemented `init_pose_publisher.cpp` so that the node published to `/initialpose` only once. This sometimes caused issues, so we modified it to publish the initial pose repeatedly until AMCL received it.
- We wanted to use a more robust approach in launching nodes (not just a timer) but due to time constraints we were not able to do it
- Similar problem to the previous one: we would like to implement a better way to run the detection of the tables but again we had not enough time to find a better solution
- In `nav2pose.cpp` we initially searched for transformation between map and Apriltags in a straight way but it always gave us "impossible to find that transform" error, but we were unable to understand why at first. We then understand that because the TF buffer had just been created and was still empty → to fix this problem we added a while loop waiting for the transformation to become available

6 Use of external material

We used draw.io for creating diagrams for this report.

LLMs were used for:

- brainstorming
- improving our understanding of what was happening and helping us diagnose some errors
- 5% of the code was directly written by LLMs

7 Appendix

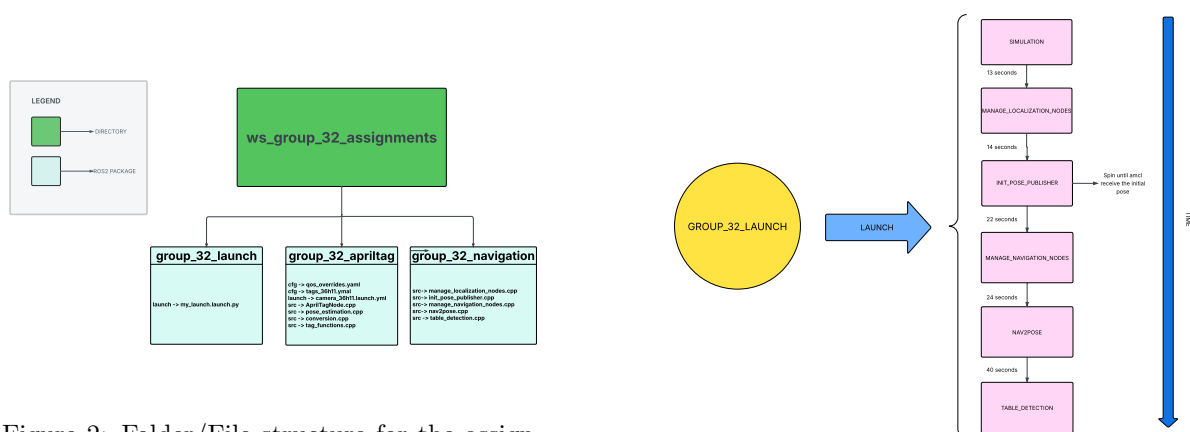


Figure 2: Folder/File structure for the assignment

Figure 3: Launch nodes sequence

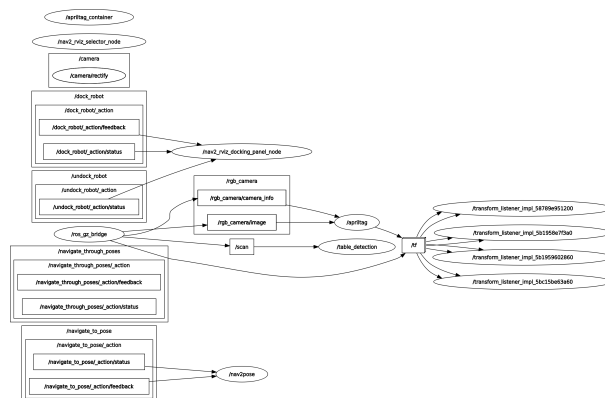


Figure 4: RQT graph

7.1 Code

```

1 for (size_t i = 1; i < laser_msg->ranges.size(); ++i)
2 {
3     // Check for discontinuity
4     auto current_distance = laser_msg->ranges[i];
5     auto previous_distance = laser_msg->ranges[i - 1];
6     if (current_distance - previous_distance < -distance_threshold)
7     {
8         possible_minimum = 1; // Potential start of a table
9         start_table_index = i;
10    }
11    else if (current_distance - previous_distance > distance_threshold && possible_minimum)
12    {
13        // Computing dimensions of the detected table
14        end_table_index = i - 1;
15        double angle_cluster = (end_table_index - start_table_index) * laser_msg->angle_increment; // Angle of the arc detecting table
16
17        // Check if the arc length is eligible to be considered a table (more robust to specific cases)
18        if (angle_cluster < M_PI/3.0)
19        {
20            table_count++;
21            // Calculation of table radius
22            int central_index = (start_table_index + end_table_index) / 2;
23            double central_distance = laser_msg->ranges[central_index];
24            double central_angle = laser_msg->angle_min + central_index * laser_msg->angle_increment;
25            double table_radius = (previous_distance * tan(angle_cluster / 2.0));
26
27            // Calculating coordinates of the table center w.r.t. the scanner reference frame
28            double x_table = (central_distance + table_radius) * cos(central_angle);
29            double y_table = (central_distance + table_radius) * sin(central_angle);
30
31            // Saving table center coordinates
32            table_centers.push_back(std::make_pair(x_table, y_table));
33        }
34        possible_minimum = 0; // Reset for next detection
35    }
36 }
37

```